Lab 06: Arrays

Learning Objectives

- Understand how to declare and initialize arrays in assembly language.
- Learn to access array elements using index registers (SI, DI).
- Practice traversing arrays with loop counters (CX) and pointer incrementation.
- Implement basic array processing tasks (sum, minimum, maximum, search, etc.).
- Strengthen understanding of conditional jumps during array comparisons.

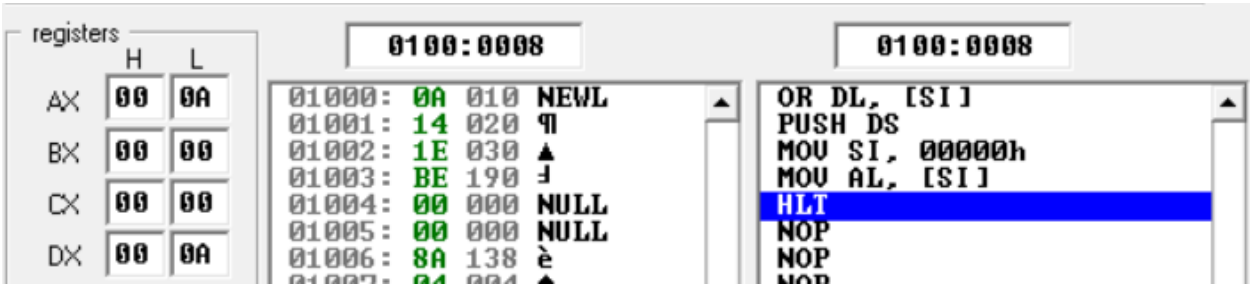
Introduction

In this lab, students will learn how arrays and strings are represented in assembly language. Arrays are stored as consecutive memory locations, and string data is simply a sequence of characters in memory. Using index registers such as **SI** (Source Index) and **DI** (Destination Index), the CPU can efficiently process arrays and strings. Mastering these concepts is essential for handling structured data and performing common tasks like copying, summing, and comparing strings.

First Program

```
ARR DB 10, 20, 30   ; Define an array

MOV SI, OFFSET ARR  ; SI points to first element
MOV AL, [SI]        ; Load first element (10) into AL

HLT
```

Output



Explanation:

- **ARR DB 10, 20, 30** → Reserves 3 bytes in memory, values: 10, 20, 30.
- **MOV SI, OFFSET ARR** → Loads the starting offset of ARR into SI (so SI points to the first element).
- **MOV AL, [SI]** → Reads the byte at DS:SI into AL; since SI = ARR, the first value (10) is copied.
- **Effective address** = DS:SI (segment:offset addressing).

Offset:

- OFFSET ARR → assembler replaces this with the numeric address (offset) of the label ARR within the data segment (DS).
- Example: if ARR starts at 0200h inside the data segment, then OFFSET ARR = 0200h.
- So after MOV SI, OFFSET ARR, you're not copying the array values — you're copying the address of the first element into SI.
- That's why SI "points" to the array start.
- When you do MOV AL, [SI], the CPU uses that address (DS:0200h) to fetch the actual value (10).

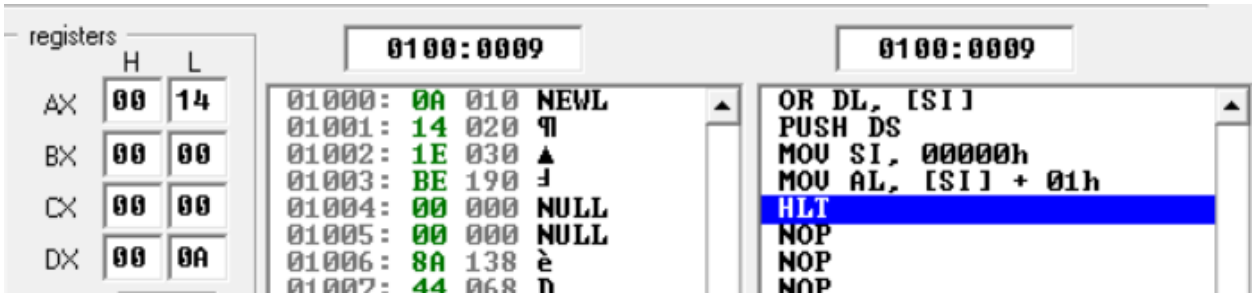
Why DS:SI?

- On the 8086, memory is accessed using segment:offset addressing; the segment comes from a segment register (e.g., DS) and the offset from another register or displacement.
- By default, [SI] is interpreted as DS:SI, meaning the offset in SI is added to the base address in DS.
- MOV SI, OFFSET ARR loads the starting offset of ARR into SI, so SI points to the first element of the array.
- MOV AL, [SI] then reads the byte at DS:SI (the first element, 10) and copies it into AL.

Accessing second element

```
ARR DB 10, 20, 30   ; Define an array

MOV SI, OFFSET ARR  ; SI points to first element
MOV AL, [SI + 1]       ; Load first element (10) into AL

HLT
```

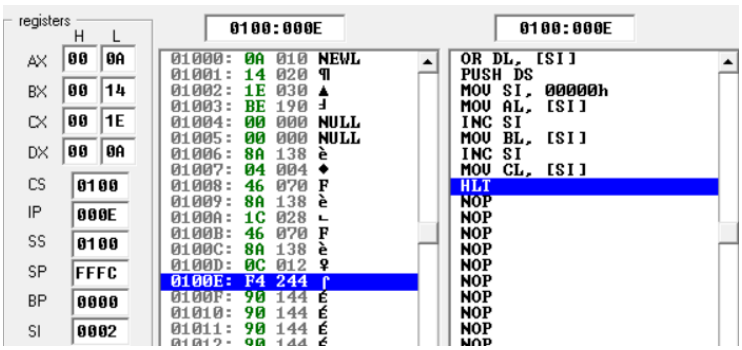
Output



Sequentially Read Array Elements

```
ARR DB 10, 20, 30   ; array in memory

MOV SI, OFFSET ARR  ; point SI to start of array

MOV AL, [SI]       ; AL = 10
INC SI             ; move to next element
MOV BL, [SI]       ; BL = 20
INC SI             ; move to next element
MOV CL, [SI]       ; CL = 30

HLT
```

Output



Difference:

- INC SI → Permanently increments SI, so future memory accesses use the new location.
- [SI+1] → Temporarily adds 1 for that instruction only; SI itself remains unchanged.

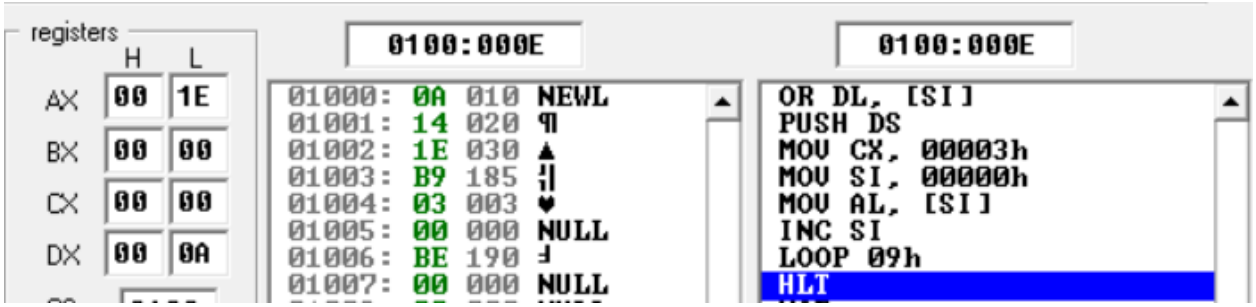
Array Traversal Using SI and LOOP

```
ARR DB 10, 20, 30      ; Array of 3 elements

MOV CX, 3              ; Counter = number of elements
MOV SI, OFFSET ARR     ; SI points to first element

NEXT:
   MOV AL, [SI]        ; Load current element into AL
   INC SI              ; Move to next element
   LOOP NEXT           ; Repeat until CX = 0

HLT
```

Explanation:

- **MOV CX, 3** → sets loop counter to number of elements.
- **MOV SI, OFFSET ARR** → SI points to the start of the array.
- **MOV AL, [SI]** → loads the current array element.
- **INC SI** → moves SI to the next element.
- **LOOP NEXT** → decrements CX automatically, continues until CX = 0.

Output



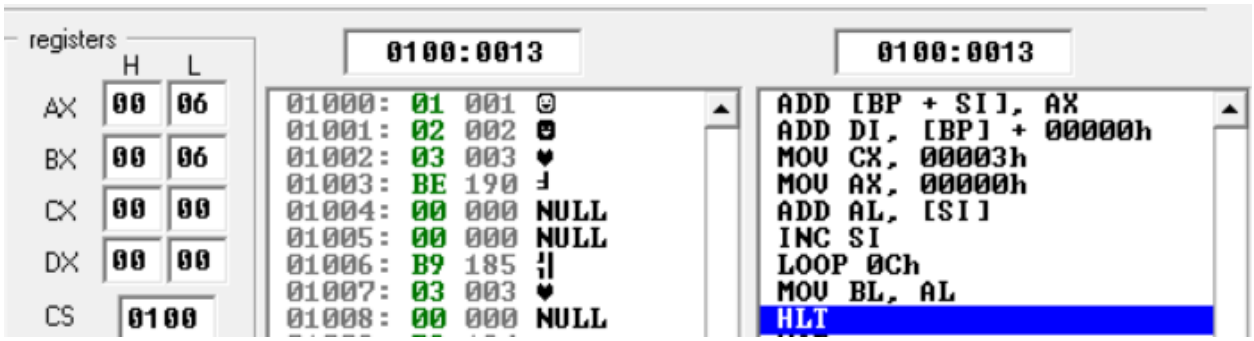
Sum of Array Elements

```
ARR DB 1, 2, 3   ; Array of 3 elements

MOV SI, OFFSET ARR  ; SI points to first element
MOV CX, 3           ; Loop counter = number of elements
MOV AX, 0           ; AX will hold the sum (start with 0)

NEXT:
   ADD AL, [SI]    ; Add current element to AL
   INC SI          ; Move to next element
   LOOP NEXT       ; Repeat until CX = 0

MOV BL, AL          ; Store final sum (for checking in BL)
HLT                 ; End program
```

Explanation

- ARR DB 1,2,3 → array in memory.
- MOV SI, OFFSET ARR → point SI to the start.
- MOV CX, 3 → loop will run 3 times.
- ADD AL, [SI] → add element to accumulator (AL).
- INC SI → move to next element.
- LOOP NEXT → continue until all elements processed.
- Result: AL = 6 (sum of 1+2+3).

Output



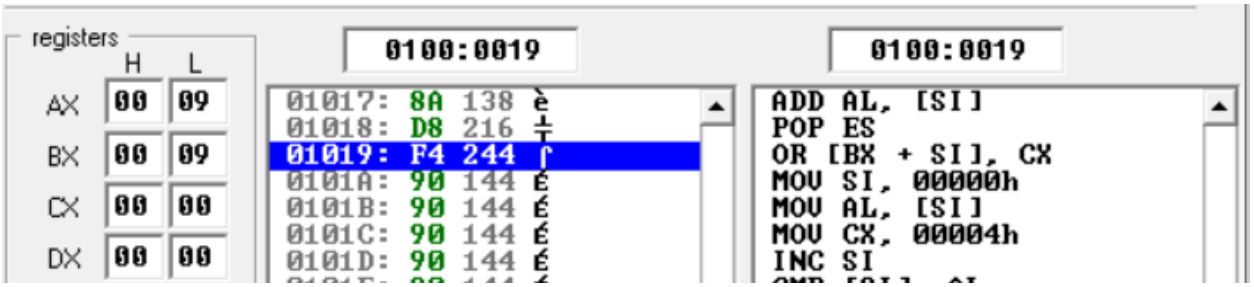
Finding Maximum

```
ARR DB 2, 4, 7, 9, 8   ; Array of 5 elements

MOV SI, OFFSET ARR      ; SI points to first element
MOV AL, [SI]            ; Assume first element is max
MOV CX, 4               ; Remaining elements = 4
INC SI                  ; Move to second element

NEXT:
   CMP [SI], AL         ; Compare current element with max
   JBE SKIP             ; If = AL, skip
   MOV AL, [SI]         ; Else update max
SKIP:
   INC SI               ; Move to next element
   LOOP NEXT            ; Repeat until CX = 0

MOV BL, AL              ; Final max stored in BL
HLT                     ; End program
```

Explanation (concise)

- ARR DB ... → defines array.
- MOV AL, [SI] → first element as initial max.
- Loop through remaining elements with CX.
- CMP + JBE → check if current ≤ max.
- If larger, update max (MOV AL, [SI]).
- At the end → maximum stored in BL.

Output



Exercise:

Task 1: Find Minimum Number – Scan array, compare elements, keep smallest in a register.

Task 2: Find Average of Array – Sum all elements, divide by count to get average.

Task 3: Find Specific Number – Traverse array, compare each element, stop if match found.